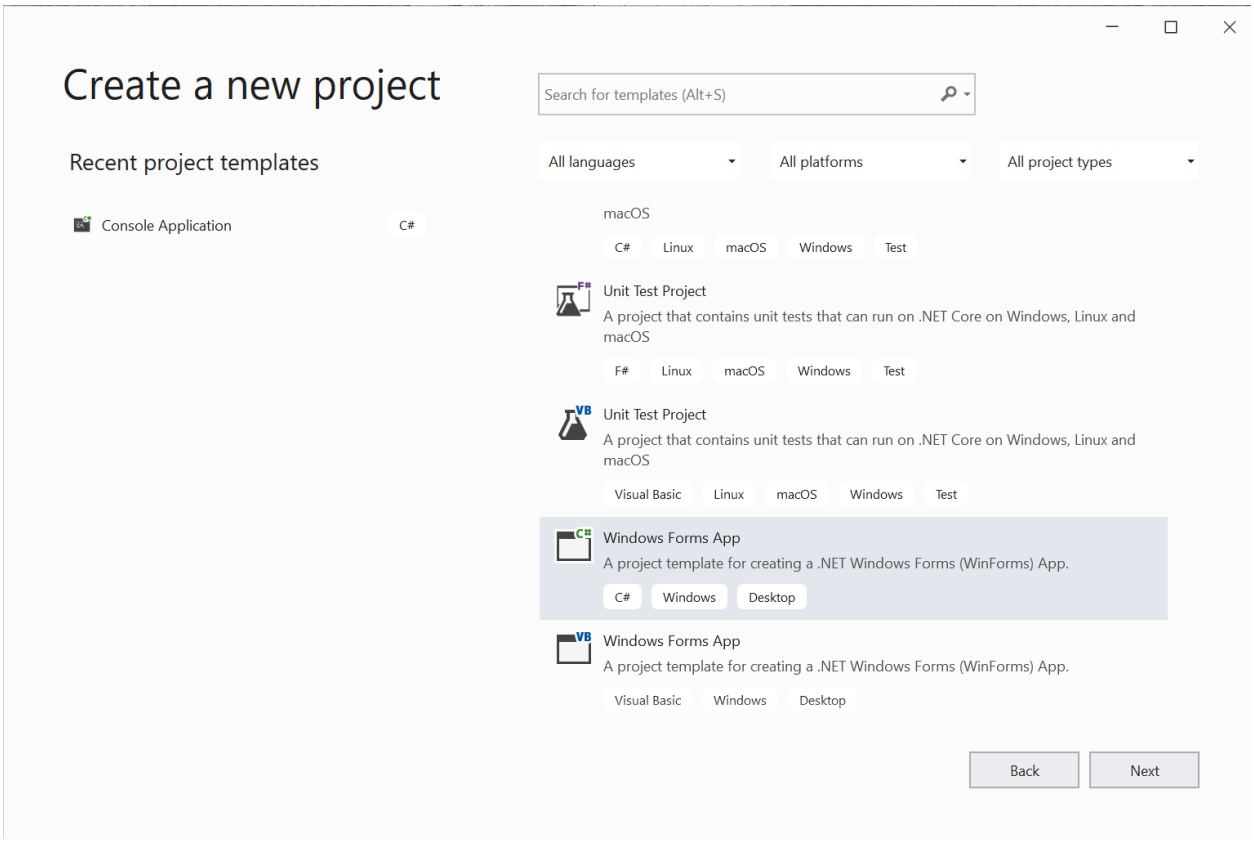
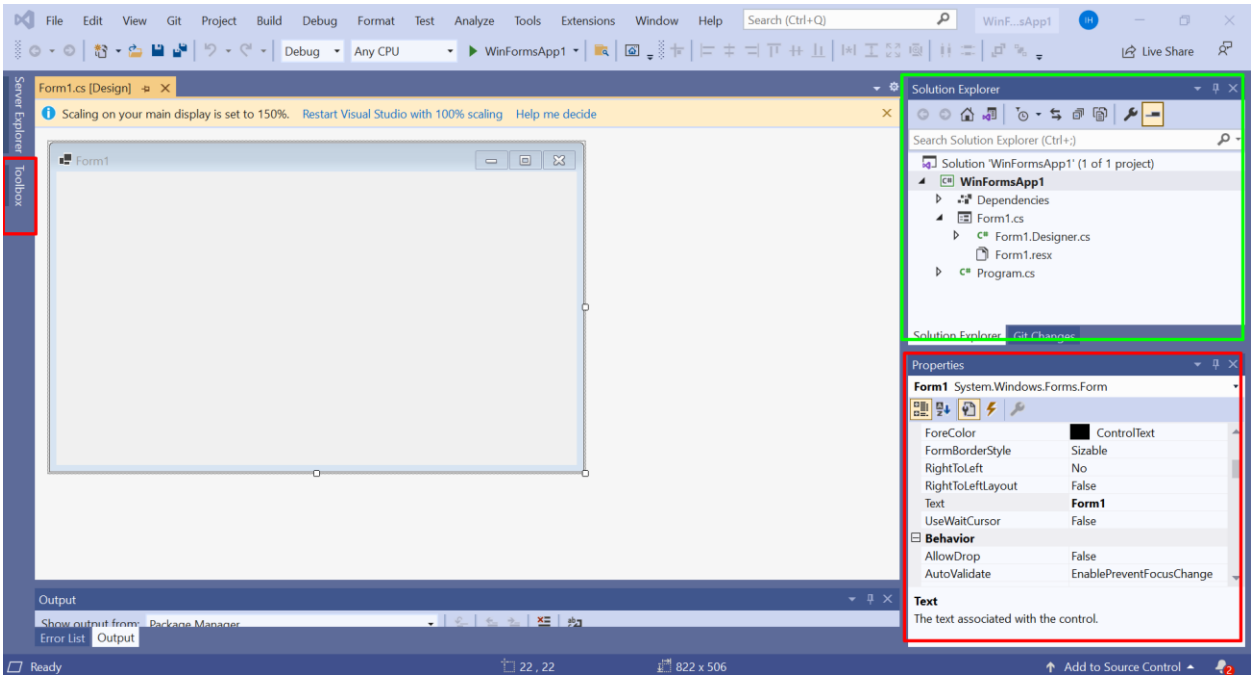
Lecture 6

Windows Form Application

Windows forms are being used for creating visual interfaces for the desktop application. They contain many controls which can be used as per the requirement. This lecture explains some of the basic controls.

Let’s start by creating a new project





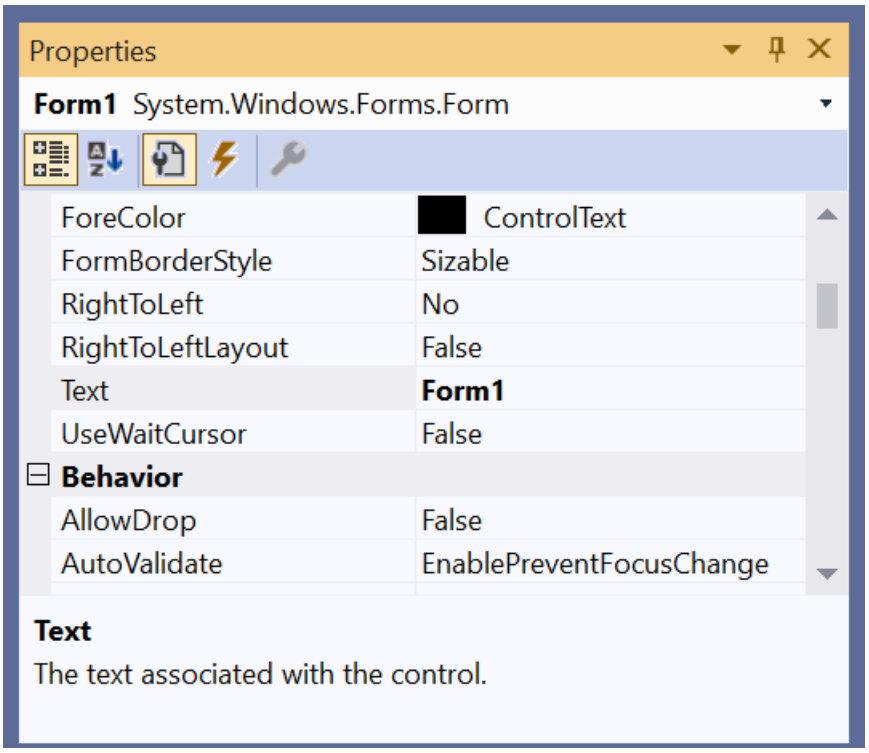
Let’s discuss one by one

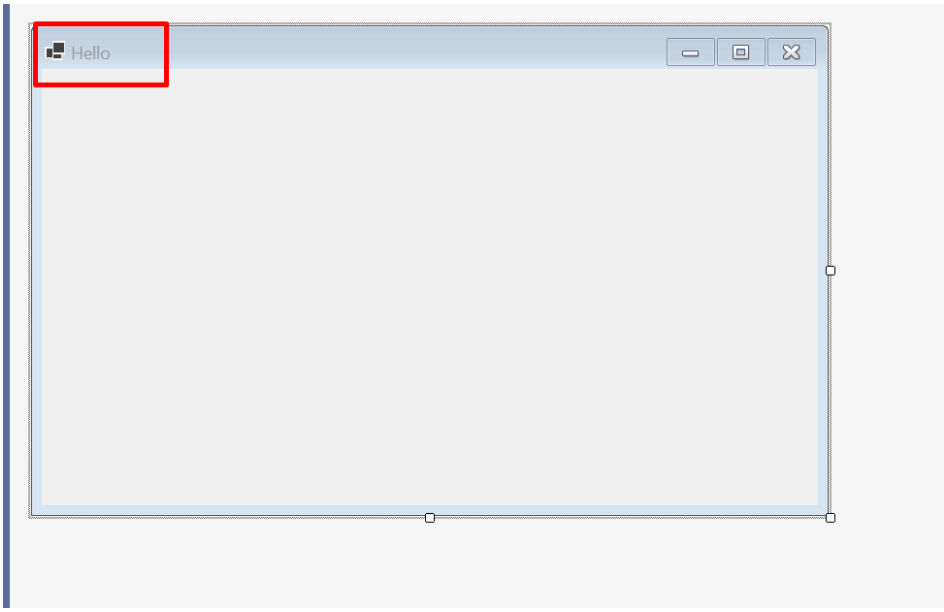
1. Designer
    a. In the Designer we see what the form looks like. So far, it's just an empty window.
2. Properties
    a. In the Properties window, we can see the properties of the currently selected element on the form. If you don't see the window, enable it in View -> Properties Window.
3. Toolbox
    a. Toolbox is a sliding window that serves as a palette with individual controls that we can add to the form.

Setting control properties

When we select an element on the form or the form itself, we can change the properties of that element in the Properties window.

Since we got no element on the form, it's the form what is selected. We'll change the title to "Hello". Find the **Text** property and enter Hello in it. The result is reflected in the designer immediately. This way we'll set the properties of all elements on the form.

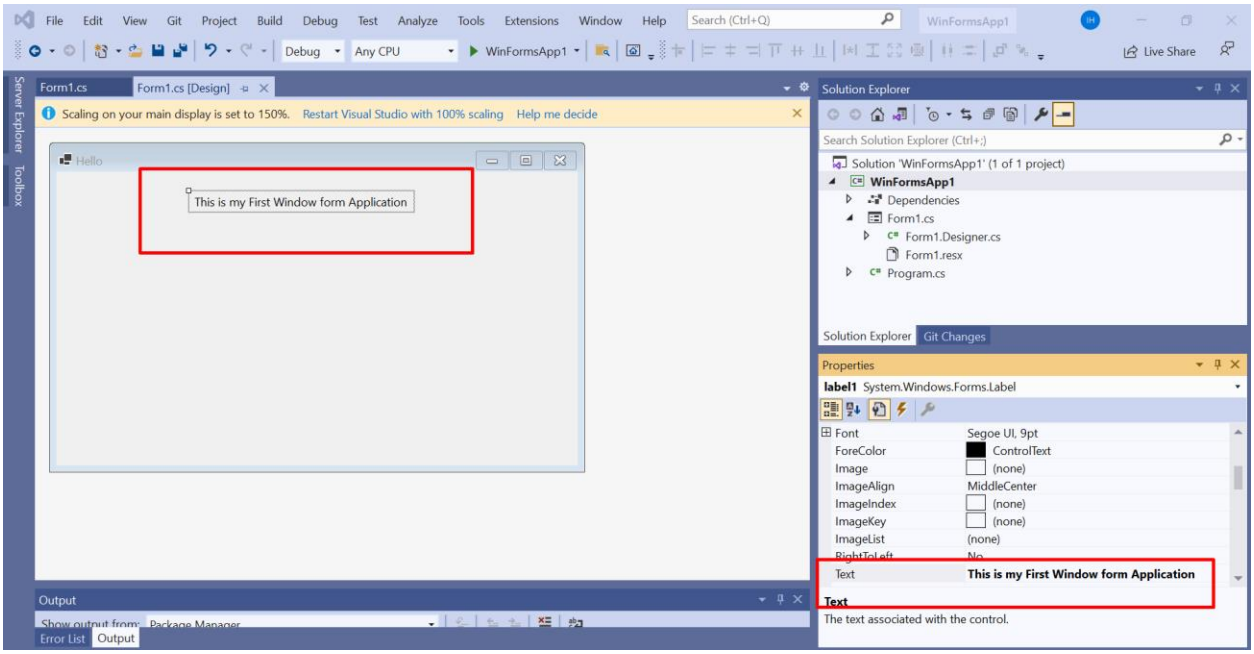




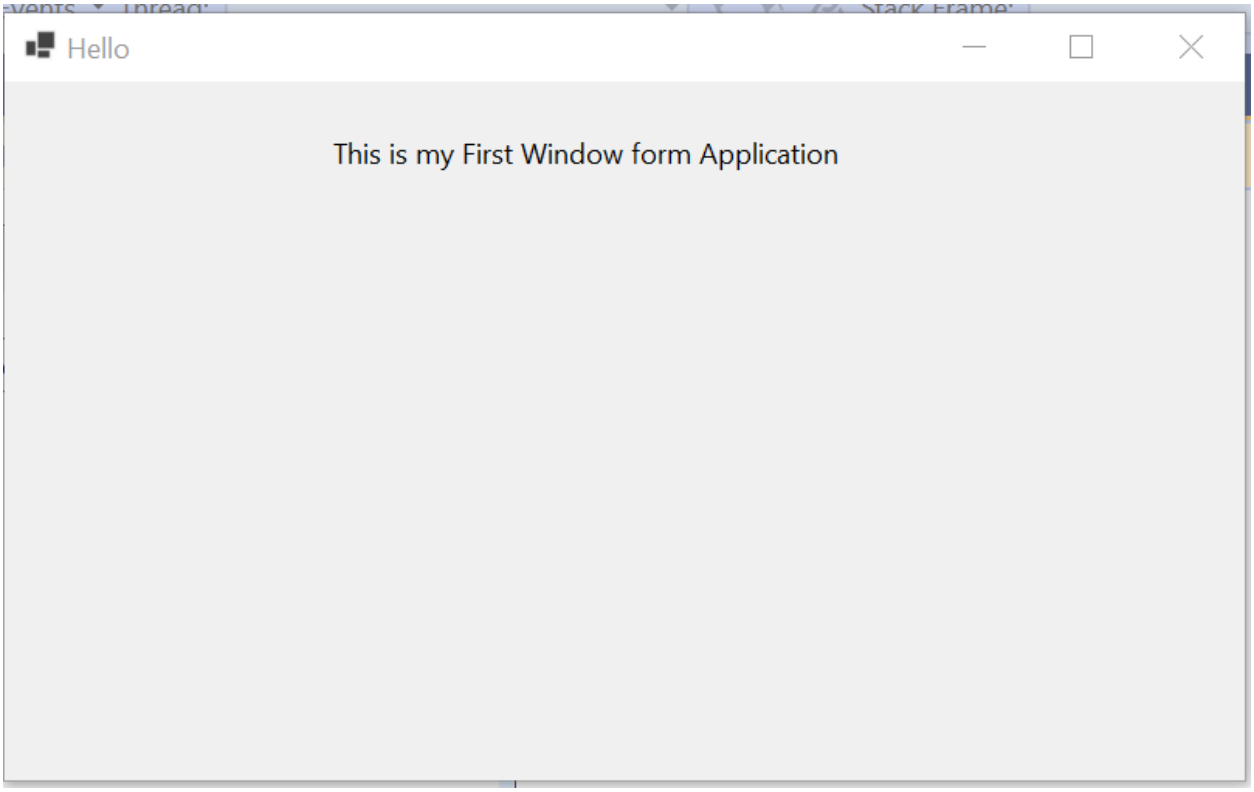
Adding Controls to the Form

Now we'll open the Toolbox and select the **Label** control, which is a text label. We'll insert it into the form either by double-clicking on it or by dragging it. Reduce the form size and simply

drag the label in the middle. In the Properties window, set the label to "Hello This is my First Window Form application".

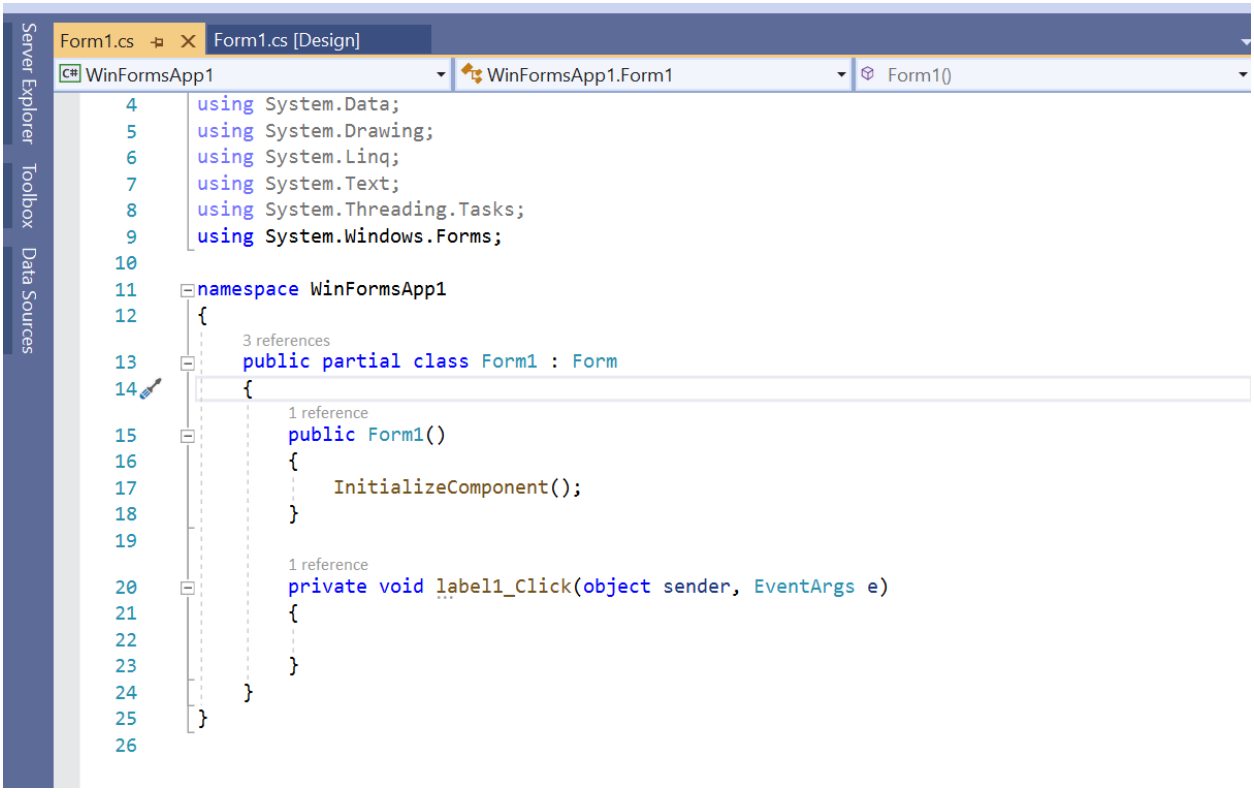


Output:



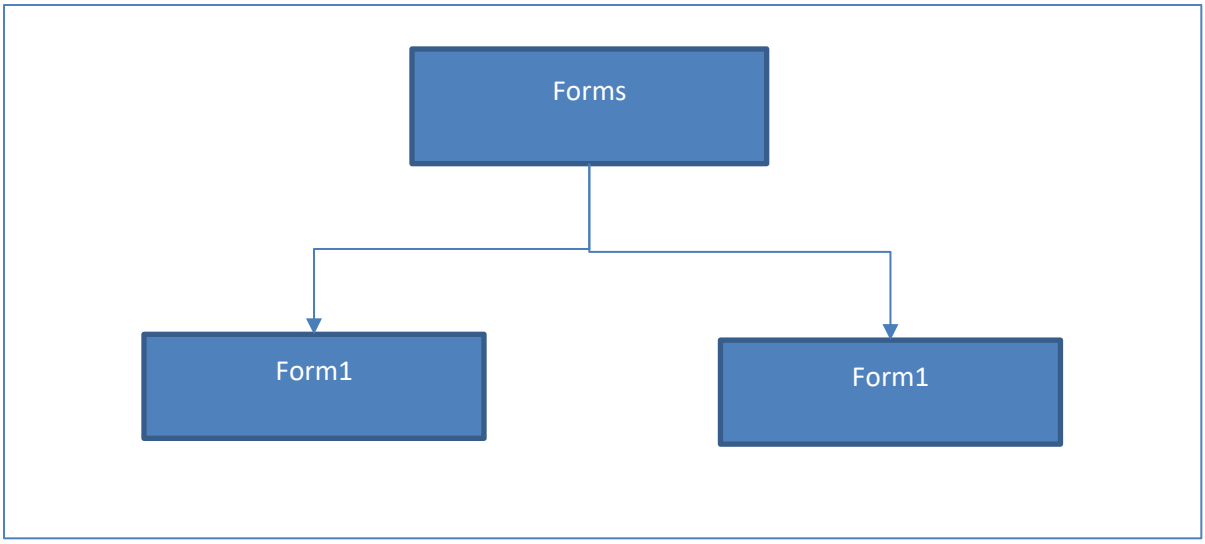
Code insights:

Double click on any object and you'll be able to see the code that is working behind the visual interface.



Partial Class:

A partial class is a special feature of C#. It provides a special ability to implement the functionality of a single class into multiple files and all these files are combined into a single class file when the application is compiled. A partial class is created by using a ***partial*** keyword.



When we execute the above code, the compiler combines Form1.cs and Form2.cs into a single file, i.e. Forms as shown below.

Form1.cs

```
public partial class Forms {
    private string Author_name;
    private int Total_articles;

    public Forms(string a, int t)
    {
            this.Authour_name = a;
            this.Total_articles = t;
    }
}
```

Form2.cs

```
public partial class Forms {
```

```
    public void Display()
    {
            Console.WriteLine("Author's name is : " + Author_name);
            Console.WriteLine("Total number articles is : " + Total_articles);
    }
}
```

Forms

```
    public class Forms {
        private string Author_name;
        private int Total_articles;

        public Forms(string a, int t)
        {
                this.Authour_name = a;
                this.Total_articles = t;
        }

        public void Display()
        {
                Console.WriteLine("Author's name is : " + Author_name);
                Console.WriteLine("Total number articles is : " + Total_articles);
        }
    }
```

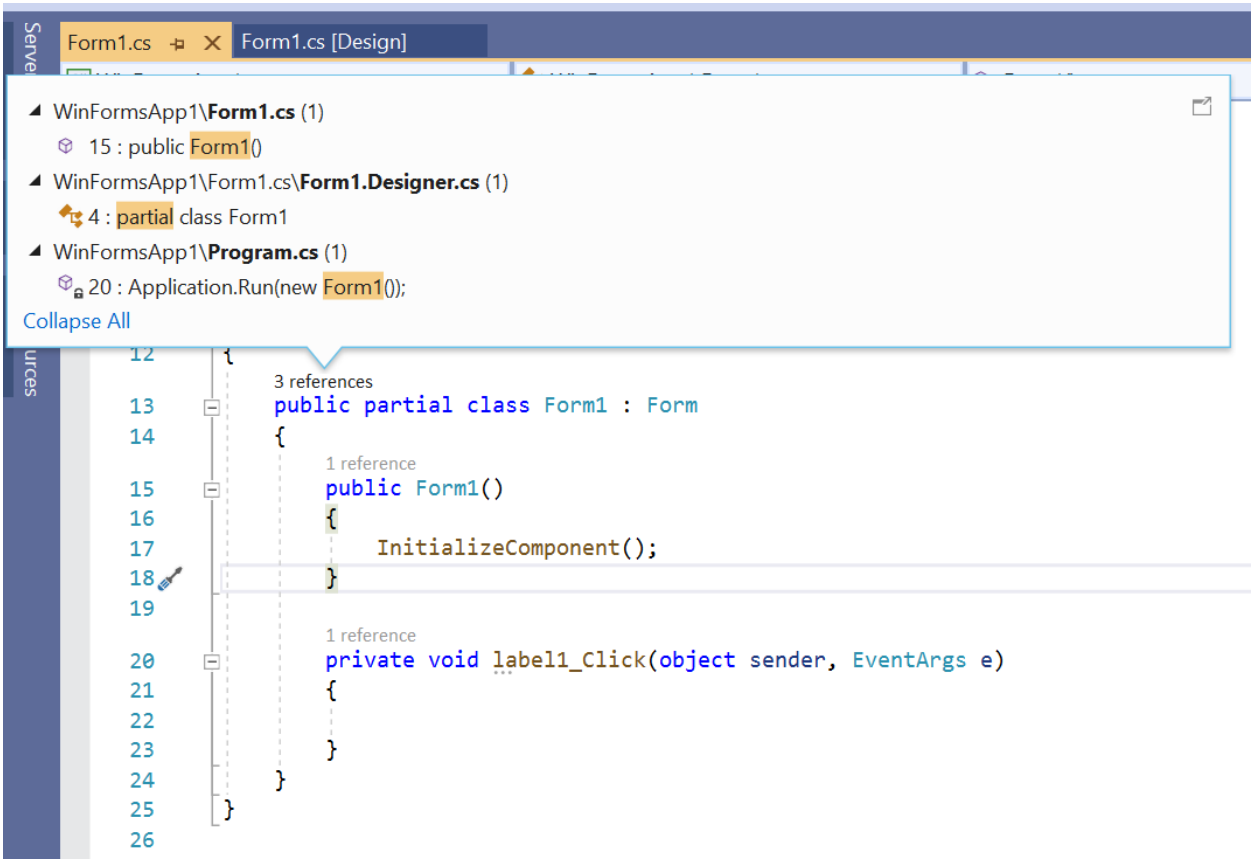
Initialize components ()

InitializeComponent is a method automatically written for you by the Form Designer when you create/change your forms.

The **InitializeComponent** method contains the code that creates and initializes the user interface objects dragged on the form surface.

Of course, due to the high numbers of changes executed by the Form Designer, it is a really good advice to **not try to modify manually this method.**

References:



Event Handlers:

Event handlers are methods in an object that are executed in response to some **events** occurring in the application. For example when you double clicked on label you will be redirected to the code below:

Which is a click event of the label.

```
        private void label1_Click(object sender, EventArgs e)
        {

        }
```

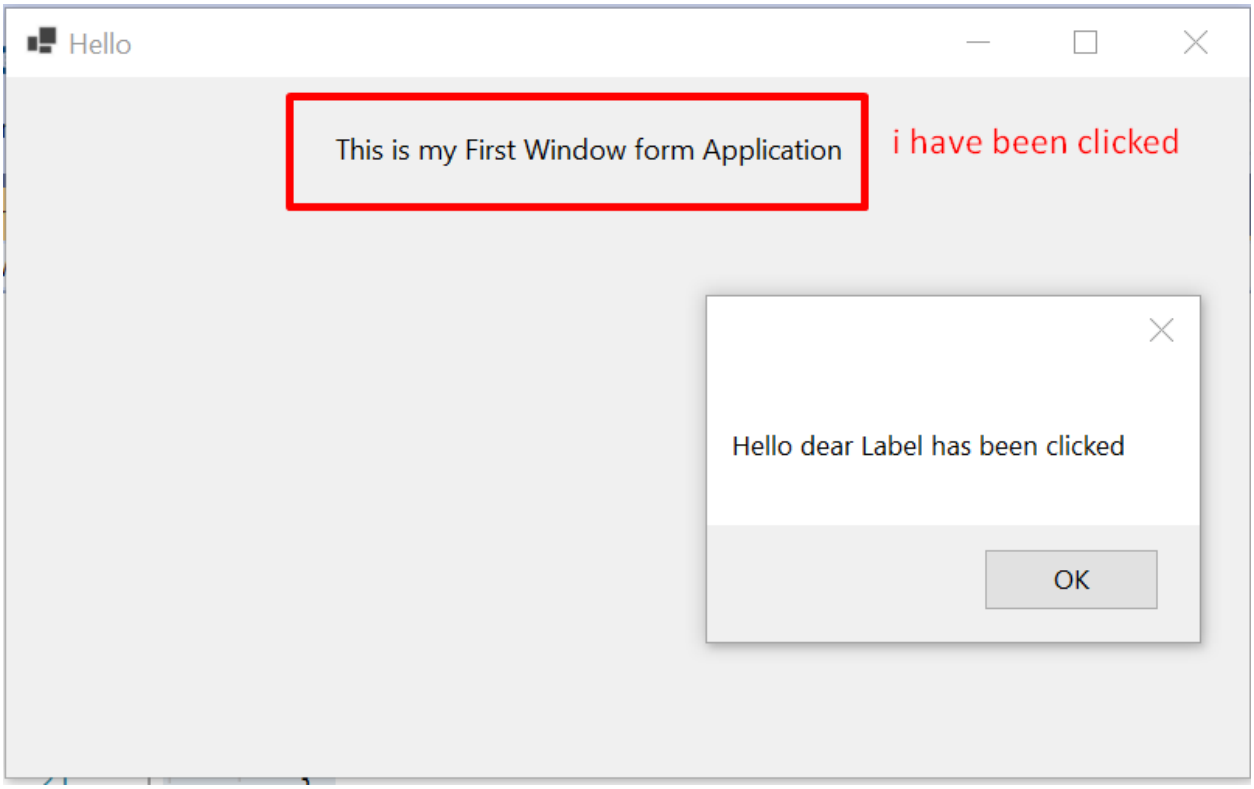
If you want to perform some task by clicking on the label you can write your code here. Let's see an example of Message Box which will appear by clicking the label that we have added in our code.

```
private void label1_Click(object sender, EventArgs e)
        {
            MessageBox.Show("Hello dear Label has been clicked");
        }
```

Output:

When you will click on label following message box will appear.

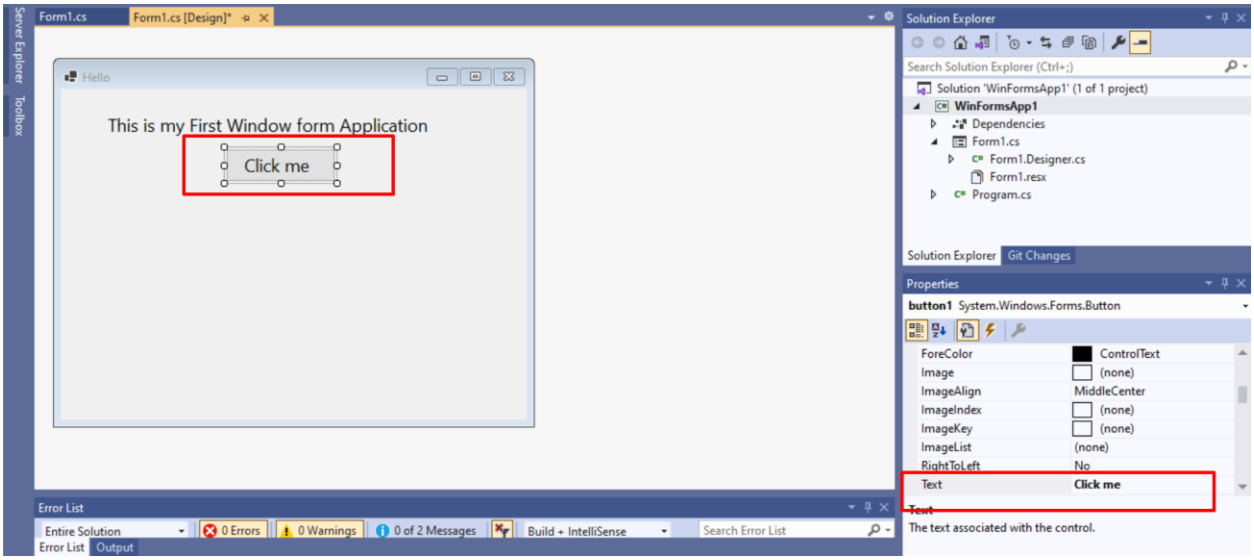
Message box is used to display some kind of message in the form a small box.



Understood the basics?

Now let's add some more elements from toolbox

Button:

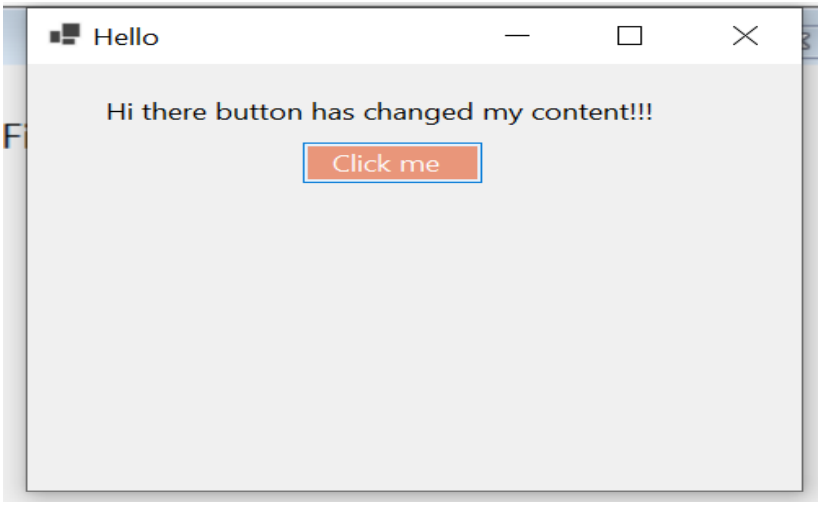


Let's change the text on button click. Button event is being created with button name.

Note: do not confuse button name and text. Name is used as id to access the object whereas, text displays the text which we want to show to the user.

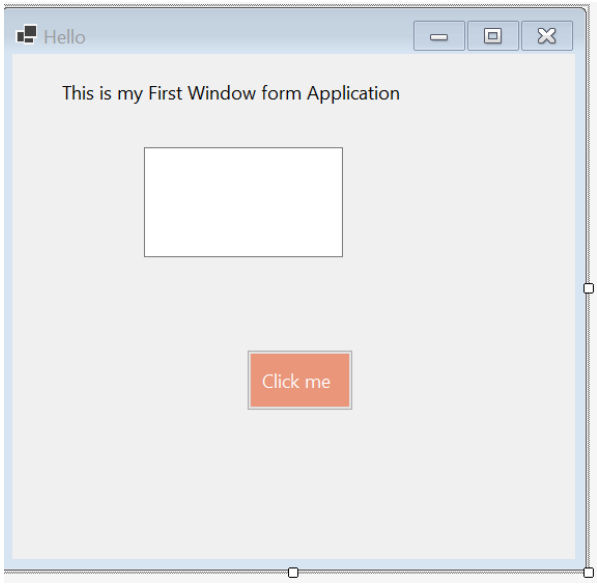
```
        private void button1_Click(object sender, EventArgs e)
        {
            label1.Text = "Hi there button has changed my content!!!";
        }
```

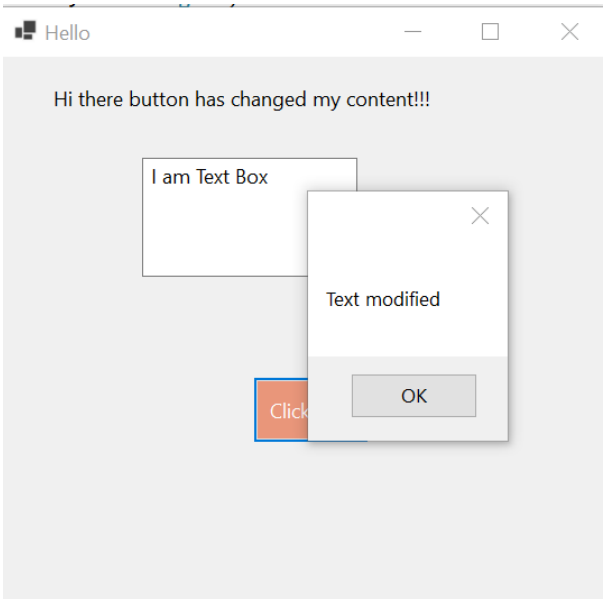
Output:



Textbox:

Used to display text or input text from user.





Code:

```
private void button1_Click(object sender, EventArgs e)
        {
            label1.Text = "Hi there button has changed my content!!!";
            textBox1.Text = "I am Text Box";
        }

        private void textBox1_TextChanged(object sender, EventArgs e)
        {
```

```
            MessageBox.Show("Text modified");
        }
```

'SENDER':

is called object which has some action perform on some control. Object Sender is a parameter called Sender that contains a reference to the control/object that raised the event.

'EVENT' :

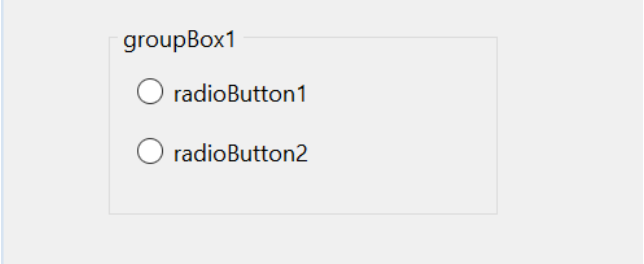
its having some information about control which has some behavior and identity perform by some user. When action will generate by occurring for event add it keep within array is called event agrs. EventArgs e is a parameter called e that contains the event data

Group Box:

Used to group items together. For example

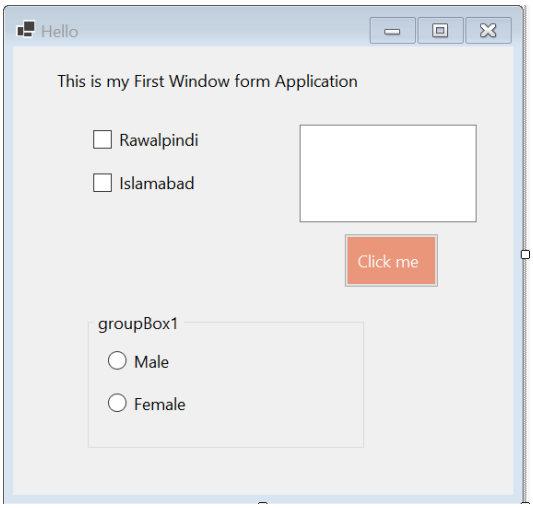
Radio Buttons:

Can chose only one option



Check Boxes:

Used to select multiple options



Lab Task:

1. Try using ListView /Listbox and add data from array.
2. Take input from user into a text box and display multiple inputs in the form a list.
3. Use data and time picker to get the date of today and display that date in label or textbox.